

Three lenses on the HLT–offline gap

A note on what each alignment loss can see — and therefore fix. Colors throughout:

teal = offline / teacher

purple = HLT / student

amber = merged / corrupted

Setup

The same jet, painted twice

Every simulated jet exists in two versions: the **offline reconstruction** — precise track parameters, well-resolved particles — and the **HLT reconstruction** — the same jet measured faster and rougher: smeared d_0 and p_T , some particles missing, some pairs of particles merged into one. Think of them as two paintings of the same scene: one careful, one done in thirty seconds with a thick brush.

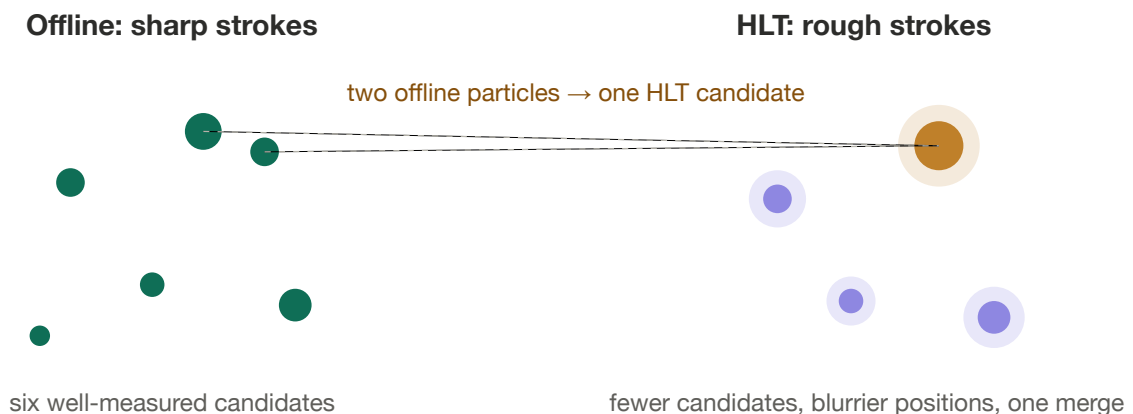


Figure 1 — Two reconstructions of the same jet. The HLT view has smeared positions (halos), a missing soft particle, and one **merged candidate** where offline sees two.

Our student network only ever receives the HLT view. During training, though, we can show it the offline view of the *same* jet and ask: make your internal representation of this jet resemble what the offline encoder produces. One hard fact bounds everything:

The ceiling. No network can recreate information the HLT reconstruction destroyed — a track that was never found has no impact parameter to recover. Alignment losses do not add information; they help the student *fully use* the information that survived. The three

losses below differ in which mistakes they can see. A gradient only fixes what its loss makes visible.

There are three kinds of mistakes to worry about, and conveniently, one loss per mistake: **smearing** (features shifted a little), **merging** (wrong particle count, corrupted relations), and **relevance** (of all remaining differences, which ones actually hurt b-tagging?).

Rung (b) — token MMD

Compare the paint, not the strokes

Embed each particle with the student or teacher encoder; each jet becomes a cloud of points in embedding space. Give every point a soft halo (a Gaussian kernel of width σ — the brush size) and sum the halos: the cloud becomes a smooth density, a painting. The token MMD is simply the squared difference between the **student painting** and the **teacher painting** :

$$MMD^2 = (\text{self-similarity of } X) + (\text{self-similarity of } Y) - 2 \times (\text{cross-similarity of } X \text{ and } Y)$$

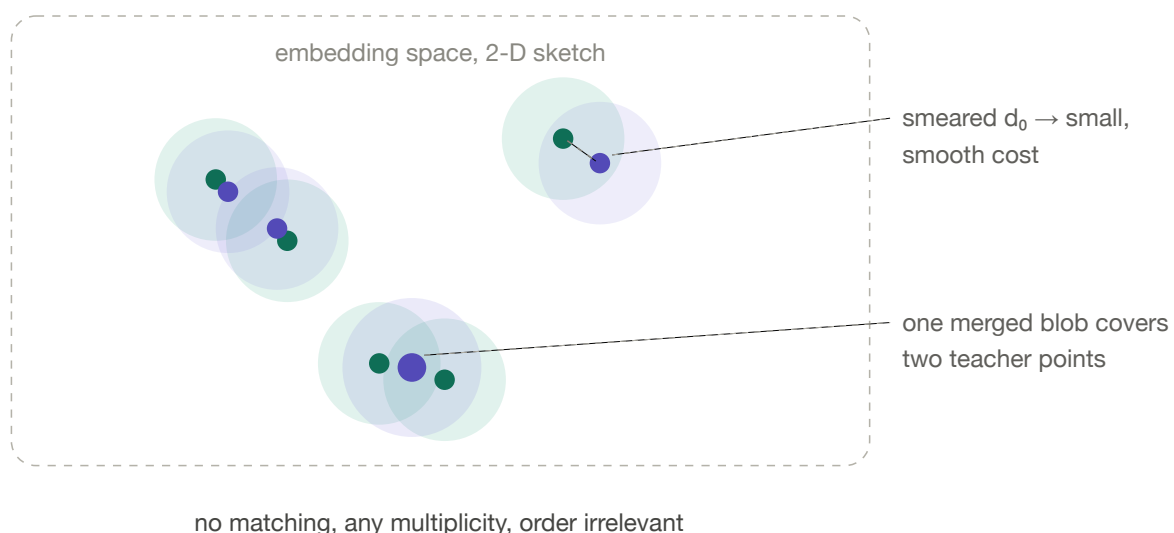


Figure 2 — The token MMD compares smoothed densities. Slightly offset points overlap their targets, so smearing produces a small cost that shrinks smoothly as the student improves. A merged blob laying its weight over two teacher points is nearly free.

What it sees well: smearing. Shifted features move the density, the loss notices, and the

gradient tells the encoder exactly which way to move — with no matching of particles ever required, so different particle counts and orderings are harmless by construction.

What it forgives: merging. That tolerance is a feature (it is why this loss trains stably where index-matching failed) but also a blind spot — the loss barely pushes the encoder to *repair* a merge, only to place its total weight correctly.

What it cannot see: relations. The painting records where particles are, not which particles agree with each other. But b-tagging is about agreement: several displaced tracks consistent with one secondary vertex.

Rung (c) — pairwise MMD

Look at the edges, not the dots

Change the object of study: for every pair of particles (i, j), compute the quantities that describe their *relationship* — opening angle ΔR , pair mass, k_T . Each pair becomes one point in a new space; a jet with n particles becomes a cloud of $n(n-1)/2$ pair-points. Then run *exactly the same MMD* on those clouds. One implementation detail matters: weight each pair by $p_{T,i} \cdot p_{T,j}$, and keep only pairs among the ~ 30 hardest particles so the pair count stays manageable.

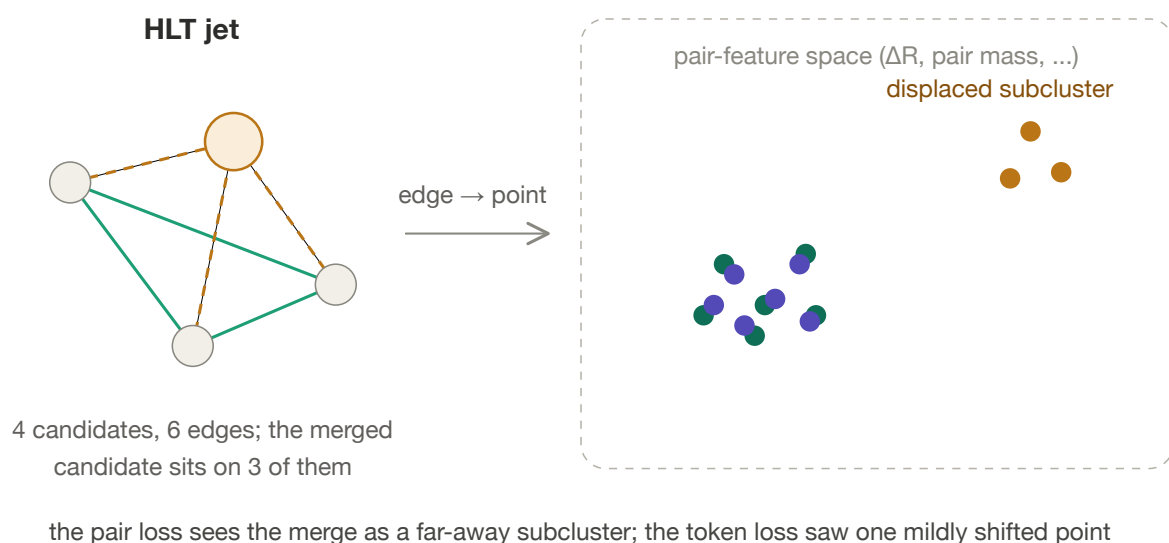


Figure 3 — Left: one merged candidate touches three of the six edges. Right: in pair-feature space those three pairs carry wrong masses and angles and land far from the teacher's bulk — a large, unmissable discrepancy.

What it sees well: merging. A single merged candidate corrupts every edge it belongs to — the error is amplified $(n-1)$ times and moved into the loss's field of view. The gradient now actively rewards the encoder for using context ("given my neighbors, I am probably two particles") to embed merged candidates the way offline embeds the resolved pair.

Why the physics wants this. The Hbb signature is relational: tracks that are displaced *and mutually consistent* with one decay vertex. The pair cloud is that relational structure, made into a first-class citizen of the training objective — and it is the same information (ΔR , k_T , pair mass) that ParT feeds to its attention as a bias.

What it still treats blindly: all pair mismatches count equally, whether or not they matter for telling Hbb from QCD.

Rung (d) — tagger-space MMD

Measure errors through the tagger's eyes

Rungs (b) and (c) use an even-handed ruler: every direction of embedding space is penalized equally. But not every direction matters. Rung (d) borrows the metric from the network that will actually consume these embeddings: pass both clouds through the *first block of the frozen, offline-trained tagger*, and compute the MMD on the activations that come out. The block's weights are never updated — but gradients flow *through* it back into the student, so the student is trained to look right *as seen by the tagger*. In computer vision this is called a perceptual loss: compare pictures as a trained network sees them, not pixel by pixel.

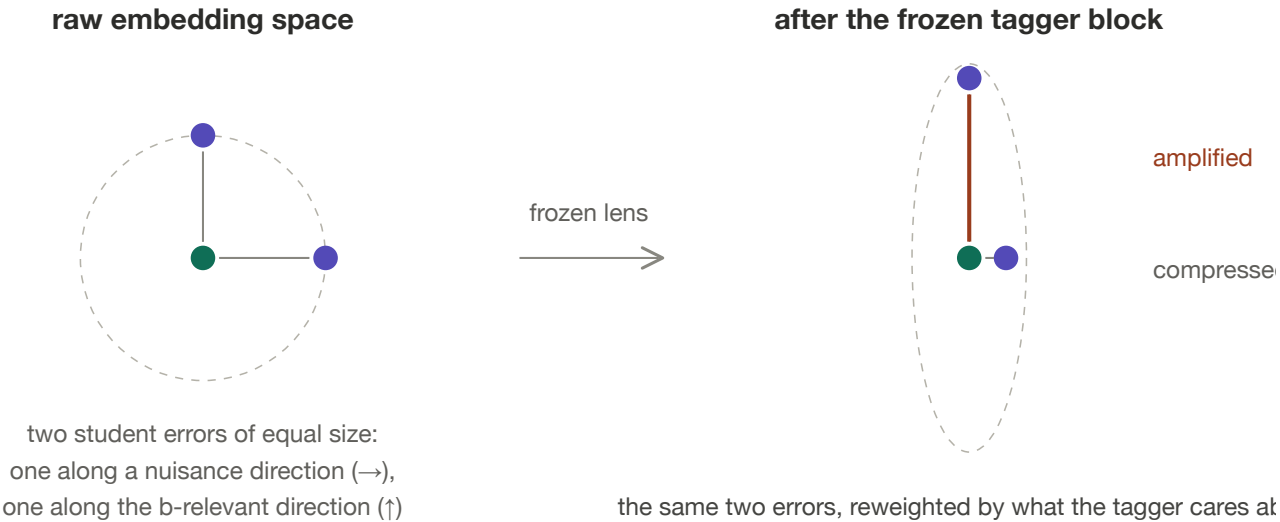


Figure 4 — Left: two student mismatches of identical size in raw embedding space. Right: seen through the frozen tagger block, the discrimination-relevant error is stretched and the nuisance error is squeezed. The MMD computed here spends the student's capacity where it buys tagging performance.

What it sees well: relevance. The tagger's early layers were trained to stretch discrimination-relevant directions and compress nuisance ones. An MMD computed after that lens automatically prioritizes exactly the residual differences that would cost Hbb-vs-QCD performance — including subtle ones nobody thought to name.

A bonus: ParT's first block already mixes particles through pair-feature-biased attention, so part of rung (c)'s relational sensitivity comes bundled in for free.

Care needed: early in training the student's tokens are out-of-distribution for the frozen block, so warm up with rung (b) for a few epochs; and align at early blocks — by the last block the representation has collapsed toward the classification output and the signal becomes as coarse as jet-level distillation.

Summary

Three nested lenses, one diagnosis

Rung	Aligns	Sees sharply	Blind or forgiving to	Analogy
(b) token MMD	which particles exist	smearing of d_0 , p_t , positions	merging (tolerated, not repaired); relations	compare two paintings

(marginals)

(c) pairwise MMD	how particles relate (correlations)	merging, corrupted pair masses and angles	whether a given mismatch matters	compare the edges, not the dots
(d) tagger-space MMD	what the tagger perceives	the residual differences that cost performance	nothing new — it reweights, it does not detect new error types	judge through the consumer's eyes

Two closing points worth internalizing. First, the losses are not three repair tools — the repairing is always done by the same student encoder. They are three *error detectors* of increasing selectivity, and a gradient can only fix what its detector reports. Second, because only the loss changes between rungs, the performance increments are physics measurements in disguise: if (c) adds a lot over (b), merging was a dominant loss mechanism; if (d) adds little over (b)+(c), the hand-built pair features already captured what matters. Running the ladder *is* diagnosing where the HLT–offline gap lives.

And always remember the ceiling. All three rungs share it. Success is not "HLT becomes offline"; success is measured against the best network trained directly on HLT inputs — the direct baseline every variant must beat.